Raport z etapu 7

Przetwarzanie dużych zbiorów danych

Szymon Jędrzejczak (266867)
Mateusz Nasewicz (268513)
Szymon Wieczorek (266586)

13.05.2026

Spis treści

1 Przegląd implementacji etapów MapReduce **2**

2 Wybrane fragmenty kodu źródłowego **2**
2.1 E2Driver – Złączenie wielu źródeł (MultipleInputs) 2
2.2 GtfsCombiner – Optymalizacja pre-agregacji 3

3 Czasy wykonania potoku MapReduce **4**

4 Eksperymenty i optymalizacja **4**
4.1 Eksperyment B: Wpływ włączenia Combinera 4
4.2 Eksperyment C: Skalowalność zadań redukujących 4

5 Logi z wykonania zadań YARN **5**

1 Przegląd implementacji etapów MapReduce

System został zaimplementowany z wykorzystaniem paradygmatu MapReduce w ekosystemie Apache Hadoop (część etapów wspierana jest także skryptami PySpark). Składa się on z kilku zdefiniowanych kroków, z których każdy odpowiada za konkretny element potoku przetwarzania danych:

- **E1.1 (Ekstrakcja i normalizacja GTFS)** – przetwarza rozkłady jazdy i łączy zbiory `stops.txt`, `stop_times.txt` z wykorzystaniem technik złączeń po stronie mappera (Map-Side Join).
- **E1.5 (Pre-agregacja GTFS)** – agreguje liczbę odjazdów w godzinach szczytu dla każdego przystanku. Wykorzystuje wyspecjalizowane mappery, reducery oraz wariantowo combiner.
- **E1.2 (Ekstrakcja i normalizacja OSM)** – przetwarza dane z GeoParquet, odfiltrowując POI i nadając im wagi. Konwersja formatów i filtrowanie zaimplementowane są częściowo w Sparku przed przejściem do natywnego Hadoopa.
- **E1.3 (Normalizacja ofert nieruchomości)** – przetwarza dane deweloperskie uprzednio wzbogacone o współrzędne geograficzne (geokodowanie realizowane wcześniej przez lokalny serwer Nominatim). Mapper odczytuje znormalizowane pliki CSV z kolumnami `lat/lon` i emituje rekordy indeksowane kluczem Geohash.
- **E2 (Obliczanie wskaźników per nieruchomość)** – jest to centralny punkt analityczny. Wykorzystuje `MultipleInputs` do łączenia znormalizowanych strumieni z przystankami (T1.5), punktami POI (T1.2) i ofertami deweloperów (T1.3) na poziomie reducera (Reduce-Side Join) przy użyciu identyfikatorów Geohash jako kluczy.
- **E3 (Macierz korelacji)** – podsumowuje przetworzone zbiory, wyliczając korelacje Pearsona pomiędzy wyznaczonymi wskaźnikami (dostępnością usług, węzłów komunikacyjnych, wskaźnikiem hałasu) a metrykami takimi jak cena czy metraż, w podziale na miasta.

2 Wybrane fragmenty kodu źródłowego

2.1 E2Driver – Złączenie wielu źródeł (MultipleInputs)

Poniższy kod przedstawia konfigurację zadania łączącego trzy różne wejściowe zbiory danych z wykorzystaniem specyficznych dla nich klas `Mapper`. Wszystkie emitują wspólny format klucz-wartość, na podstawie którego odbywa się później *Reduce-Side Join*.

```
1  package gtfs;
2
3  import org.apache.hadoop.conf.Configuration;
4  import org.apache.hadoop.fs.Path;
5  import org.apache.hadoop.io.Text;
6  import org.apache.hadoop.mapreduce.Job;
7  import org.apache.hadoop.mapreduce.lib.input.SequenceFileInputFormat;
8  import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
9  import org.apache.hadoop.mapreduce.lib.output.TextOutputFormat;
10 import org.apache.hadoop.mapreduce.lib.input.MultipleInputs;
11
12 public class E2Driver {
13
14     // args: <t1_5_dir> <t1_2_dir> <t1_3_dir> <output_t2> [walkingRadiusM]
15     public static void main(String[] args) throws Exception {
16         if (args.length < 4) {
17             System.err.println("Użycie: E2Driver <t1_5> <t1_2> <t1_3> <output_t2> [radiusM]");
18             System.exit(1);
19         }
20
21         double radius = args.length >= 5 ? Double.parseDouble(args[4]) : 800.0;
22         Configuration conf = new Configuration();
23         conf.setDouble("pipeline.walking.radius.m", radius);
24
25         Job job = Job.getInstance(conf, "E2 Indicators per Property");
26         job.setJarByClass(E2Driver.class);
27
```

```
        // MultipleInputs: każde źródło ma własny Mapper
        MultipleInputs.addInputPath(job, new Path(args[0]),
                SequenceFileInputFormat.class, JoinMapperT15.class);
        MultipleInputs.addInputPath(job, new Path(args[1]),
                SequenceFileInputFormat.class, JoinMapperT12.class);
        MultipleInputs.addInputPath(job, new Path(args[2]),
                SequenceFileInputFormat.class, JoinMapperT13.class);

        // Wspólne typy wyjścia Mappera
        job.setMapOutputKeyClass(Text.class);
        job.setMapOutputValueClass(JoinRecord.class);

        job.setReducerClass(E2Reducer.class);
        job.setNumReduceTasks(3);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(Text.class);
        job.setOutputFormatClass(TextOutputFormat.class);

        FileOutputFormat.setOutputPath(job, new Path(args[3]));
        if (!job.waitForCompletion(true)) System.exit(1);
    }
}
```

2.2 GtfsCombiner – Optymalizacja pre-agregacji

Zaimplementowany combiner optymalizuje przesyłanie danych pomiędzy mapperem a reducerem, sumując wartości lokalnie w ramach tego samego węzła, jednak *bez* wprowadzania modyfikacji do samego klucza sortowania/partycjonowania.

```
package gtfs;

import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;
import java.io.IOException;

public class GtfsCombiner extends Reducer<Text, StopInfo, Text, StopInfo> {

    private final StopInfo combined = new StopInfo();

    @Override
    protected void reduce(Text key, Iterable<StopInfo> values, Context context)
            throws IOException, InterruptedException {

        double totalScore = 0;
        boolean isFirst = true;

        for (StopInfo info : values) {
            totalScore += info.score;
            if (isFirst) {
                combined.lat    = info.lat;
                combined.lon    = info.lon;
                combined.geoId  = info.geoId;
                combined.stopId = info.stopId;
                isFirst = false;
            }
        }

        combined.score = totalScore;
        // klucz NIE jest modyfikowany - zostaje stopId (oryginalne key)
        context.write(key, combined);
    }
}
```

3 Czasy wykonania potoku MapReduce

Wyniki uzyskane podczas uruchomienia pełnego potoku danych zaobserwowane na klastrze testowym zebrano w tabeli 3.1. Czas trwania etapu E1.2 dominuje z uwagi na kosztowną konwersję plików GeoParquet oraz ekstrakcję i wstępne filtrowanie danych wektorowych obejmujących cały kraj w środowisku PySpark. Czasy zostały uśrednione z 3 pełnych iteracji uruchomienia potoku MapReduce (pominięto czas przesyłania i aktualizacji początkowych danych do HDFS).

Tabela 3.1: Czasy wykonania poszczególnych etapów potoku dla 3 uruchomień oraz czas średni

Etap	Opis operacji	Run 1 [s]	Run 2 [s]	Run 3 [s]	Średni czas [s]
E1.1	Ekstrakcja i normalizacja GTFS	20	19	20	20
E1.5	Pre-agregacja GTFS (odjazdy/przystanek)	22	23	23	23
E1.2	Konwersja OSM Parquet do znormalizowanych struktur	218	219	213	217
E1.3	Ekstrakcja i normalizacja ofert (częściowo cache)	64	64	62	63
E2	Obliczanie wskaźników per nieruchomość (łączenie T1.5, T1.2, T1.3)	74	76	73	74
E3	Wyznaczanie wskaźników korelacji (ceny względem metryk)	21	22	20	21
Łączny średni czas wykonania zadań analitycznych					**418**

4 Eksperymenty i optymalizacja

W ramach prac nad potokiem wykonano pomiary wydajnościowe na wycinkach infrastruktury (agregacja danych). Analizowano wpływ włączenia fazy Combiner w architekturze MapReduce oraz sterowania stopniem zrównoleglenia zadań (liczba reducerów).

4.1 Eksperyment B: Wpływ włączenia Combinera

Porównano czas pre-agregacji (etap E1.5) dla jednego zadania reducer w wariancie z użyciem i bez użycia dedykowanego combinera (`GtfsCombiner`). Zmierzono zarówno czas wykonania, jak i rozmiar danych przesyłanych w fazie shuffle.

Tabela 4.1: Wyniki eksperymentu B – Combiner

Wariant	Run 1 [s]	Run 2 [s]	Run 3 [s]	Średni czas [s]	Rozmiar shuffle [MB]
Combiner wyłączony	18	19	21	19.3	44.1
Combiner włączony	19	20	20	19.6	0.85

Wniosek: Mimo że czasy wykonania obu wariantów są zbliżone, włączenie combinera drastycznie redukuje ilość danych przesyłanych w fazie shuffle. Zbiór danych dla pojedynczego miasta (GTFS ZTM) jest na tyle niewielki, że narzut inicjalizacji lokalnej agregacji niweluje zysk czasowy. Jednak znacząca redukcja shuffla pokazuje, że przy większej skali danych (np. ogólnokrajowa sieć komunikacyjna) combiner przyniesie wyraźną poprawę wydajności.

4.2 Eksperyment C: Skalowalność zadań redukujących

Poddano badaniu czas wykonania etapu E1.5 bez użycia combinera przy zmianie liczby zadań klasy Reducer (od 1 do 3).

Tabela 4.2: Wyniki eksperymentu C – Liczba reducerów

Liczba reducerów	Run 1 [s]	Run 2 [s]	Run 3 [s]	Średni czas [s]
1	19	20	20	19.6
2	18	20	21	19.6
3	19	20	21	20.0

Wniosek: Równoległe przetwarzanie w fazie reduce dla 2 i 3 procesów na zadanym zbiorze (GTFS) nie przełożyło się na poprawę czasu całkowitego. Brak zysku czasowego potwierdza asymetrię w obciążeniu (często obserwowaną dla nierównomiernie rozłożonych miast z jednym silnym klastrem, np. Warszawa) oraz uwydatnia narzut czasowy związany z koordynacją cyklu życia samych kontenerów zadaniowych YARN w stosunku do krótkiego czasu obliczeń właściwych dla relatywnie niewielkiego zbioru przystanków.

5 Logi z wykonania zadań YARN

Poniżej zamieszczono surowe logi dla bardziej czasochłonnego i rozbudowanego etapu **E2** (Obliczanie wskaźników per nieruchomość) z trzeciego uruchomienia potoku. Pokazano prawidłowe łączenie z ResourceManagerem, ładowanie wielu źródeł danych (`MultipleInputs` widać po liczbie *Total input files to process*), długotrwały postęp wykonania faz mapowania i redukcji, a także wylistowanie niestandardowych liczników biznesowych (np. `CELLS_WITH_OFFERS`, `POIS_IN_CELL`) obok standardowych liczników wbudowanych w framework MapReduce po zakończeniu zadania ze statusem *completed successfully*.

```
2026-05-13 11:24:40,327 INFO client.DefaultNoHARMFailoverProxyProvider: Connecting to
↪  ResourceManager at master/10.0.2.4:8050
2026-05-13 11:24:40,480 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing
↪  not performed. Implement the Tool interface and execute your application with ToolRunner to
↪  remedy this.
2026-05-13 11:24:40,487 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path:
↪  /tmp/hadoop-yarn/staging/root/.staging/job_1778661823181_0030
2026-05-13 11:24:40,699 INFO input.FileInputFormat: Total input files to process : 12
2026-05-13 11:24:40,702 INFO input.FileInputFormat: Total input files to process : 1
2026-05-13 11:24:40,727 INFO input.FileInputFormat: Total input files to process : 179
2026-05-13 11:24:40,767 INFO mapreduce.JobSubmitter: number of splits:192
2026-05-13 11:24:41,217 INFO mapreduce.JobSubmitter: Submitting tokens for job:
↪  job_1778661823181_0030
2026-05-13 11:24:41,313 INFO impl.YarnClientImpl: Submitted application
↪  application_1778661823181_0030
2026-05-13 11:24:41,329 INFO mapreduce.Job: The url to track the job:
↪  http://master:8088/proxy/application_1778661823181_0030/
2026-05-13 11:24:41,329 INFO mapreduce.Job: Running job: job_1778661823181_0030
2026-05-13 11:24:45,361 INFO mapreduce.Job:  map 0% reduce 0%
2026-05-13 11:24:51,591 INFO mapreduce.Job:  map 3% reduce 0%
2026-05-13 11:25:01,923 INFO mapreduce.Job:  map 20% reduce 0%
2026-05-13 11:25:11,167 INFO mapreduce.Job:  map 42% reduce 0%
2026-05-13 11:25:16,251 INFO mapreduce.Job:  map 52% reduce 11%
2026-05-13 11:25:21,463 INFO mapreduce.Job:  map 64% reduce 16%
2026-05-13 11:25:26,591 INFO mapreduce.Job:  map 78% reduce 21%
2026-05-13 11:25:31,770 INFO mapreduce.Job:  map 92% reduce 26%
2026-05-13 11:25:34,791 INFO mapreduce.Job:  map 100% reduce 44%
2026-05-13 11:25:38,802 INFO mapreduce.Job:  map 100% reduce 67%
2026-05-13 11:25:41,809 INFO mapreduce.Job:  map 100% reduce 100%
2026-05-13 11:25:42,816 INFO mapreduce.Job: Job job_1778661823181_0030 completed successfully
2026-05-13 11:25:42,857 INFO mapreduce.Job: Counters: 64
        File System Counters
                FILE: Number of bytes read=195879396
                FILE: Number of bytes written=445985260
                HDFS: Number of bytes read=128718996
                HDFS: Number of bytes written=189899289
        Job Counters
```

```
                Killed map tasks=1
                Launched map tasks=192
                Launched reduce tasks=3
                Data-local map tasks=187
                Rack-local map tasks=5
                Total time spent by all maps in occupied slots (ms)=1039997
                Total time spent by all reduces in occupied slots (ms)=122310
                Total time spent by all map tasks (ms)=1039997
                Total time spent by all reduce tasks (ms)=122310
                Total vcore-milliseconds taken by all map tasks=1039997
                Total vcore-milliseconds taken by all reduce tasks=122310
                Total megabyte-milliseconds taken by all map tasks=1064956928
                Total megabyte-milliseconds taken by all reduce tasks=125245440
        Map-Reduce Framework
                Map input records=1610646
                Map output records=1664190
                Map output bytes=192550998
                Map output materialized bytes=195882834
                Input split bytes=47808
                Reduce input groups=31516
                Reduce shuffle bytes=195882834
                Reduce input records=1664190
                Reduce output records=1493462
                Spilled Records=3328380
                Shuffled Maps =576
                Merged Map outputs=576
                GC time elapsed (ms)=54821
                CPU time spent (ms)=142430
                Physical memory (bytes) snapshot=62899171328
                Virtual memory (bytes) snapshot=506855763968
                Total committed heap usage (bytes)=90254606336
                Peak Map Physical memory (bytes)=719380480
                Peak Map Virtual memory (bytes)=2606473216
                Peak Reduce Physical memory (bytes)=553193472
                Peak Reduce Virtual memory (bytes)=2622337024
        E2
                CELLS_WITH_OFFERS=139
                OFFERS_PROCESSED=1493462
                POIS_IN_CELL=1777
                STOPS_IN_CELL=4740
        E2_T12
                RECORDS_EMITTED=110491
        E2_T13
                RECORDS_EMITTED=1493462
        E2_T15
                NEIGHBOR_RECORDS_EMITTED=53544
                RECORDS_EMITTED=6693
        File Input Format Counters
                Bytes Read=0
        File Output Format Counters
                Bytes Written=189899289
```